

Week 6 – Services, Daemons and Network Management

Topics covered

- `systemd` and Linux daemons
- Service units
- Checking, starting and stopping services
- Enabling services at boot
- Viewing network interfaces and routes
- Checking listening ports
- Changing the hostname
- Managing NetworkManager connections with `nmcli`
- Configuring static IPv4 addresses, gateways and DNS

The Week 6 slides contain two labs:

- RH124 Chapter 9.5: **Control Services and Daemons**
- RH124 Chapter 11.11: **Managing Network**

The final slide also provides practice questions for both topics.

1. Understanding `systemd` and daemons

`systemd` manages the Linux startup process and controls system services.

A **daemon** is a process that normally runs in the background to perform a system task. Daemon names often end with `d`.

Examples:

Daemon/service	Purpose
<code>sshd</code>	Provides SSH remote access

<code>crond</code>	Runs scheduled tasks
<code>firewalld</code>	Manages firewall rules
<code>cups</code>	Provides printing services
<code>NetworkManager</code>	Manages network connections

A service is represented by a systemd unit ending in:

`.service`

Example:

`sshd.service`

Other unit types mentioned in the slides include:

Unit type	Purpose
<code>.service</code>	Manages a system service
<code>.socket</code>	Monitors a communication socket
<code>.path</code>	Activates a service after a file-system change

2. Listing services

List active service units:

```
systemctl list-units --type=service
```

List active and inactive service units:

```
systemctl list-units --type=service --all
```

List all installed service unit files:

```
systemctl list-unit-files --type=service
```

The output of `list-units` includes:

Column	Meaning
UNIT	Unit name
LOAD	Whether the unit file loaded correctly
ACTIVE	High-level service state
SUB	More detailed service state
DESCRIPTION	Description of the service

`list-units` shows units loaded into memory, while `list-unit-files` shows installed unit files and whether they are enabled or disabled.

3. Checking a service

View detailed information:

```
systemctl status sshd.service
```

Important fields include:

Field	Meaning
Loaded	Whether the service configuration loaded
Active	Whether the service is running
Main PID	Main process ID
Docs	Relevant manual pages
CGroup	Processes belonging to the service

Press:

```
q
```

to exit the status display.

Check only whether it is running:

```
systemctl is-active sshd.service
```

Possible output:

```
active
```

Check whether it starts automatically at boot:

systemctl is-enabled sshd.service

Possible output:

enabled

4. Service states

State	Meaning
active (running)	Service is running continuously
active (exited)	One-time service completed successfully
active (waiting)	Service is waiting for an event
inactive	Service is not running
enabled	Configured to start at boot
disabled	Not configured to start at boot
static	Cannot be enabled directly but may be started by another unit

Important: `active` and `enabled` do not mean the same thing.

A service can be:

- Active but disabled

- Inactive but enabled
 - Active and enabled
 - Inactive and disabled
-

5. Starting and stopping services

Start a service now:

```
sudo systemctl start sshd.service
```

Stop a service now:

```
sudo systemctl stop sshd.service
```

Restart a service:

```
sudo systemctl restart sshd.service
```

Reload its configuration:

```
sudo systemctl reload sshd.service
```

Difference:

- **restart** stops and starts the service, usually creating a new PID.
- **reload** rereads the configuration without completely restarting, so the main PID normally remains unchanged.

Not every service supports **reload**.

6. Enabling and disabling services

Enable a service to start automatically during boot:

```
sudo systemctl enable sshd.service
```

Disable automatic startup:

```
sudo systemctl disable sshd.service
```

Enable and start immediately:

```
sudo systemctl enable --now sshd.service
```

Disable and stop immediately:

```
sudo systemctl disable --now sshd.service
```

The slides explicitly use separate `enable` and `start` operations. The `--now` form combines them into one command.

7. Service command summary

Command	Purpose
<code>systemctl status UNIT</code>	Show detailed service status
<code>systemctl start UNIT</code>	Start a service now
<code>systemctl stop UNIT</code>	Stop a service now
<code>systemctl restart UNIT</code>	Stop and start a service
<code>systemctl reload UNIT</code>	Reload its configuration
<code>systemctl enable UNIT</code>	Start automatically at boot
<code>systemctl disable UNIT</code>	Do not start automatically
<code>systemctl is-active UNIT</code>	Check whether it is running

```
systemctl is-enabled  
UNIT
```

Check whether it starts at
boot

```
systemctl  
list-dependencies  
UNIT
```

Show required units

8. Viewing network interfaces

Display network interface information:

```
ip address
```

Short form:

```
ip addr
```

Another short form:

```
ip a
```

The output shows:

- Interface name
- Interface state
- MAC address
- IPv4 address
- IPv6 address
- Prefix length

Example:

```
1. 2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP>
```

```
inet 10.0.2.15/24
```

Here:

- Interface: `enp0s3`
- IPv4 address: `10.0.2.15`

- Prefix: /24

Display only link-layer information:

```
ip link
```

9. Interface states

Common interface states include:

State	Meaning
UP	Interface is administratively enabled
DOWN	Interface is disabled
LOWER_UP	Physical or virtual link is detected
NO-CARRIER	No network link is detected

An interface may be administratively UP but still lack an IP address.

10. Viewing the routing table

Use:

`ip route`

Example:

2. default via 10.0.2.2 dev enp0s3

10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15

The first line means:

- Default gateway: 10.0.2.2
- Outgoing interface: enp0s3

The **default route** is used when the destination does not match a more specific route.

11. Adding and deleting routes

The slides demonstrate adding and deleting routes from the routing table.

Add a route:

```
sudo ip route add 172.16.0.0/16 dev enp0s8
```

Add a route through a gateway:

```
sudo ip route add 172.16.0.0/16 via 192.168.1.1
```

Delete the route:

```
sudo ip route del 172.16.0.0/16
```

Routes added directly with `ip route` are normally temporary and may disappear after reboot unless added through a persistent NetworkManager connection.

12. Testing network connectivity

Test whether another host is reachable:

```
ping 192.168.1.1
```

Send only four packets:

```
ping -c 4 192.168.1.1
```

Possible successful result:

4 packets transmitted, 4 received, 0% packet loss

Test the loopback interface:

```
ping -c 4 127.0.0.1
```

Test DNS resolution and external connectivity:

```
ping -c 4 example.com
```

Interpretation:

Result	Possible meaning
IP ping works, hostname fails	DNS issue
Gateway ping fails	Local network or gateway issue
Loopback fails	Local network stack problem
External IP works	Routing works
All pings fail	Interface, address or route issue

13. Viewing listening ports

Use:

```
ss -tln
```

Options:

Option	Meaning
<code>-t</code>	TCP
<code>-l</code>	Listening sockets
<code>-n</code>	Display numerical ports

Include the process information:

```
sudo ss -tlnp
```

List listening TCP and UDP ports:

```
sudo ss -tulnp
```

Example:

```
LISTEN 0 128 0.0.0.0:22
```

This means a service is listening on TCP port 22 on all IPv4 interfaces.

14. Viewing and changing the hostname

Show the current hostname:

```
hostname
```

Display detailed hostname information:

```
hostnamectl
```

Change the hostname:

```
sudo hostnamectl set-hostname student-PC
```

Verify:

3. hostname

hostnamectl

The new hostname may also appear in the terminal prompt after opening a new shell or logging in again.

15. NetworkManager

NetworkManager manages network devices and connection profiles.

A **device** is the physical or virtual network interface.

Examples:

4. enp0s3

enp0s8

A **connection** is a saved configuration applied to a device.

A single device may have multiple connection profiles, but only one connection can be active on the device at a time.

Connection files are stored under:

/etc/NetworkManager/system-connections/

They normally use the suffix:

.nmconnection

16. Checking NetworkManager

Check whether NetworkManager is running:

systemctl status NetworkManager

Check devices:

nmcli device status

Example:

```
5. DEVICE TYPE STATE CONNECTION
6. enp0s3 ethernet connected Wired connection 1
7. lo loopback connected lo
```

```
enp0s8 ethernet disconnected --
```

List all saved connections:

```
nmcli connection show
```

List only active connections:

```
nmcli connection show --active
```

17. Displaying a specific connection

First list connection names:

```
nmcli connection show
```

Then display one connection:

```
nmcli connection show "Wired connection 1"
```

Important fields include:

- `connection.id`
- `connection.interface-name`
- `connection.autoconnect`
- `ipv4.method`
- `ipv4.addresses`
- `ipv4.gateway`
- `ipv4.dns`

Connection names containing spaces must be placed inside quotation marks.

18. Creating a static connection

The slides provide this network information:

- IP address: 192.168.2.100
- Prefix: /24
- Gateway: 192.168.2.1

A suitable command is:

```
8. sudo nmcli connection add \
9. type ethernet \
10. con-name static-connection \
11. ifname enp0s8 \
12. ipv4.method manual \
13. ipv4.addresses 192.168.2.100/24 \

    ipv4.gateway 192.168.2.1
```

Replace `enp0s8` with the interface used in your VM.

Add DNS:

```
14. sudo nmcli connection modify static-connection \

    ipv4.dns 8.8.8.8
```

Activate it:

```
sudo nmcli connection up static-connection
```

Verify:

```
15. ip address show enp0s8
16. ip route

nmcli connection show --active
```

The slides state that `nmcli connection add` fails when the chosen connection name already exists.

19. Modifying an existing connection

The slides demonstrate changing a connection to:

- Address: 192.0.2.2/24
 - Gateway: 192.0.2.254
- ```
17. sudo nmcli connection modify static-connection \
```

```
18. ipv4.method manual \
19. ipv4.addresses 192.0.2.2/24 \

ipv4.gateway 192.0.2.254
```

Configure automatic activation during boot:

```
20. sudo nmcli connection modify static-connection \

connection.autoconnect yes
```

Apply the changes:

```
sudo nmcli connection up static-connection
```

The slides specifically note that after modifying a connection, you need to run `nmcli connection up CONNECTION_NAME` to make the new configuration effective.

---

## 20. Configuring DNS

Configure one DNS server:

```
21. sudo nmcli connection modify static-connection \

ipv4.dns 8.8.8.8
```

Configure several DNS servers:

```
22. sudo nmcli connection modify static-connection \

ipv4.dns "8.8.8.8 1.1.1.1"
```

Apply:

```
sudo nmcli connection up static-connection
```

Check the saved DNS value:

```
nmcli connection show static-connection | grep ipv4.dns
```

---

## 21. Activating and deactivating connections

Activate:



```
sudo nmcli connection up static-connection
```

Deactivate:

```
sudo nmcli connection down static-connection
```

Delete:

```
sudo nmcli connection delete static-connection
```

Be careful not to deactivate the connection currently being used for a remote SSH session.

---

## 22. DHCP connection configuration

Set a connection to obtain addressing automatically:

```
23. sudo nmcli connection modify "Wired connection 1" \
```

```
 ipv4.method auto
```

Remove manually configured addresses:

```
24. sudo nmcli connection modify "Wired connection 1" \
```

```
25. ipv4.addresses "" \
```

```
 ipv4.gateway ""
```

Activate it:

```
sudo nmcli connection up "Wired connection 1"
```

The slides mainly focus on static addressing, so this DHCP conversion is supplementary rather than one of the listed final-slide exercises.

---

## 23. Hostname resolution

Linux normally checks hostname sources in the order configured in:

```
/etc/nsswitch.conf
```

A common entry is:

```
hosts: files dns
```

This means:

1. Check `/etc/hosts`
2. If no match is found, query DNS

The local host file is:

`/etc/hosts`

Example entry:

`192.168.2.100 server1`

The DNS resolver settings are stored or represented through:

`/etc/resolv.conf`

Important directives include:

`26. search example.com`

`nameserver 8.8.8.8`

- `search` specifies domains to try with a short hostname.
- `nameserver` specifies a DNS server.
- Up to three `nameserver` entries may be listed.